MSAGLJS: Browsing Large Graphs with Tile Pyramids and Sleeve Routing

Anonymous author(s)
Anonymous affiliation(s)

Abstract

We present a new way to visualize a large graph in the style of online geographic maps. The method builds a *tile pyramid* for *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on those maps.

The edges are routed by a method we call *sleeve routing*, which computes for each edge the shortest path in its homotopy class around the node obstacles, using the Constrained Delaunay Triangulation. We apply several heuristics to speed up the routing.

We implemented our approach in the WebGL renderer of MSAGLJS, an open-source TypeScript library for graph visualization in web browsers, with the entire pipeline running client-side, without a dedicated server. We target graphs with at most 40K nodes and 100K edges. The renderer's demo can be seen at `https://microsoft.github.io/msagljs/renderer-webgl/index.html`.

MSAGLJS is available on GitHub[1] and as NPM packages[2].

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,[3] and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS[4] is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme with semantic zoom.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [2]. It supports the layered Sugiyama scheme [44], Pivot MDS [14], and IPSep-CoLa [21], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the nodes do not overlap. The rendering pipeline builds a hierarchy of tiles—analogous to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers`

[1] `https://github.com/microsoft/msagljs`
[2] `@msagl/core`, `@msagl/drawing`, `@msagl/parser`, `@msagl/renderer-svg`, `@msagl/renderer-webgl` — all on `https://www.npmjs.com/`.
[3] In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2^{32} bytes).
[4] `https://github.com/microsoft/msagljs`

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).
Editors: TBD; Article No. 0; pp. 0:1–0:13
Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

package, giving smooth pan and zoom. Live demos are available online: a WebGL renderer for large graphs[5] and an SVG renderer for smaller, editable graphs[6].

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *sleeve routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

2.0.0.1 Graph visualization tools.

Graphviz [25, 5] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [7]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [39] that did not have the features described here. On the web, Sigma.js [8] and Cytoscape.js [23] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [6] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [1] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [38] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [41] handles huge graphs with tiles and edge bundling [34, 32] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [12], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [24], which distort a multilevel layout [27, 33] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [31] and force-directed bundling [32] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our sleeve algorithm walks the dual of a Constrained Delaunay Triangulation [43] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [36, 15, 26, 4]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [30] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [20] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [18] but requires a simple containing polygon. yWorks [11] offers "Organic edge routing" based on force-directed simulation. The approach of Dwyer and Nachmanson [22] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [45]. Our sleeve method eliminates the spanner entirely and routes

[5] `https://microsoft.github.io/msagljs/renderer-webgl/index.html`
[6] `https://microsoft.github.io/msagljs/renderer-svg/index.html`

Algorithm 1 BUILDTILEPYRAMID($G, \mathcal{C}, minTileSize$)

$T_0 \leftarrow$ single tile holding all of G clipped to its bounding box
$levels \leftarrow [T_0]$; $z \leftarrow 0$
while some tile in $levels[z]$ exceeds $\mathcal{C}$ elements **and** its width/height $\geq minTileSize$ **do**
    $levels[z{+}1] \leftarrow$ split every tile of $levels[z]$ into four sub-tiles
    within each parent, distribute nodes/labels/arrowheads by bbox, and split each edge clip at the tile midlines (Section 3.3)
    $z \leftarrow z + 1$
end while
$Z \leftarrow z$; $V_Z \leftarrow V(G)$; $s_v^{(Z)} \leftarrow 1$ for all $v \in V_Z$
compute PageRank(G)
for $z = Z - 1$ **down to** 0 **do**
    $(V_z, \{s_v^{(z)}\}) \leftarrow$ SELECTTOPKWITHADAPTIVESCALE(V_{z+1}, 2^{Z-z}) ▷ Section 3.1
    build a CDT from the inflated polylines of V_z at scales $\{s_v^{(z)}\}$
    route every edge $e \in E_z$ on this CDT, optionally using the level-($z{+}1$) route as a sleeve hint ▷ Section 3.2
    clip each edge of the level with the tiles
end for
return *levels*

directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm integrated with a browser-side tile pyramid that reroutes edges per level.

2.0.0.3 CDT-based pathfinding.

The closest precedent to our routing pipeline is the CDT-and-funnel pathfinder of Demyen and Buro [17], originally designed for game-AI units of nonzero radius. They run admissible A* search on the CDT dual (with a reduced-graph variant that collapses corridor chains into single abstract edges) to recover provably shortest paths. Their search is, however, per-query: the bounds depend on the start and goal, so it cannot share work across the edges with the same source as our per-source Dijkstra does. For the graphs we target this would be roughly an order of magnitude slower; we therefore trade their proven optimality for speed.

3 Tiling

The input to tiling is a laid-out graph together with routed edges: we prefer route the edges with sleeve routing (Section 4), to achieve the visual stability on a level change.The output is a sequence of $Z{+}1$ levels: level Z is the finest, and level 0 is the coarsest. Each level is a collection of tiles indexed by integer (x, y) grid coordinates. The output is consumed by the `TileLayer` of the `@deck.gl/geo-layers` package [10]. Pseudocode 1 describes the algorithm building the tile pyramid.

Conceptually, each tile is associated with a rectangular viewport and holds some graph elements that lie within it: the single tile at level 0 is associated with a viewport equal to the graph bounding box. Each coarser level keeps a prefix of the PageRank-ordered nodes together with all edges whose endpoints both lie in that prefix; the finest level keeps the full node and edge sets.

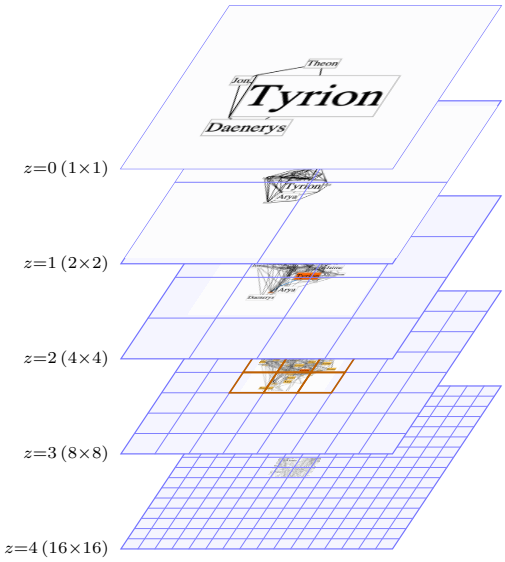


Figure 1 The entities rendered on each level as the user zooms in shown without scaling.

3.0.0.1 How the index of the finest layer, Z, is chosen.

We treat the tiles differently when calculating Z and when preparing for rendering. There is no element filtering in the former case: the top tile contains all elements of the graph, and when we split a tile we do not "lose" any elements. We continue increasing Z, generating new levels, until a stopping condition is met. Then, while keeping the finest level intact, we rebuild the tiles of levels $0, \ldots, Z-1$ for rendering.

We start from $Z = 0$ and grow the pyramid one level at a time. We stop as soon as any of three conditions is met:

1. every tile at the new level contains at most $\mathcal{C}$ elements, with default $\mathcal{C} = 500$;
2. the new tile width and height are both below ten times the average node width and height;
3. while generating the tiles of the new finest level, the running total of stored tile elements summed over all levels, multiplied by 200 bytes per element, exceeds the memory budget $\mathcal{M}_{\max}$, with default 4 GB; the partial level is then discarded and Z is set to the previous finest level.

The third condition is checked incrementally inside the level-build loop, so we abort as soon as the memory barrier is reached rather than finishing a level we know we will throw away. We additionally cap $Z \leq 8$ to bound the depth of the pyramid; the nine-level limit is comfortably above what is reachable on any input we tested.

3.0.0.2 Three phases.

Once Z and the tile rectangles are fixed, each coarser level $z = Z-1, Z-2, \ldots, 0$ is built in three phases:

1. pick the nodes of level z with PageRank and possibly enlarge them, Section 3.1;
2. reroute the edges of G_z around the resulting obstacles, Section 3.2;
3. recompute the per-tile edge clips by splitting the new curves at tile boundaries, Section 3.3.

Figure 2 shows two adjacent pyramid levels built by this procedure; Algorithm 1 summarizes the whole pipeline.

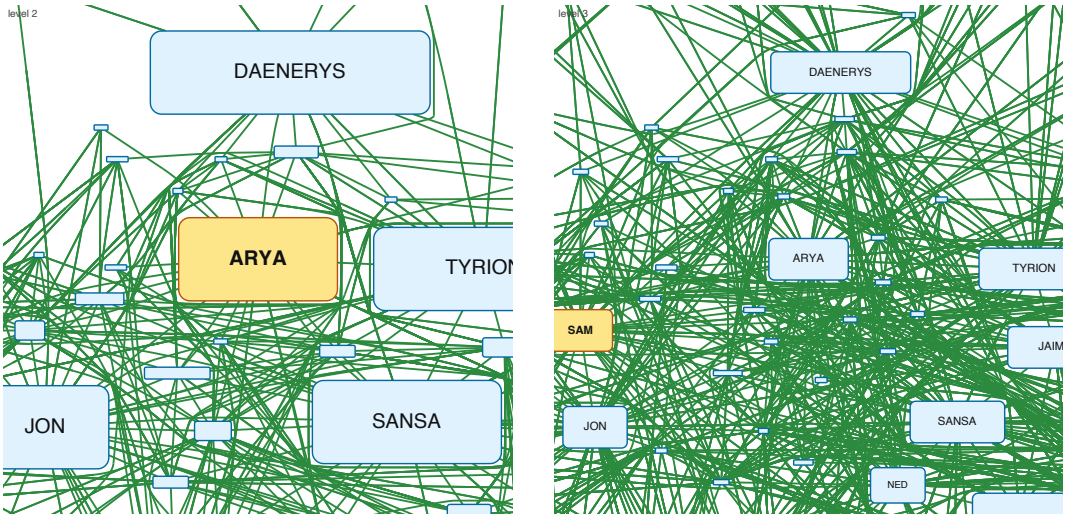


Figure 2 Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes.

3.1 PageRank-Guided Level Construction

Before filling the level's tiles for rendering we run PageRank [40] on G once, and sort all nodes of the graph, V, by descending rank. Write $k = Z - z$ for the level's depth below the finest level. The candidates for level of depth k are the nodes of the *prefix* of the sorted array of length $\lceil |V|/2^k \rceil$.

Node positions are inherited from the original layout and never move: across levels we change only each node's display size.

We add nodes to a level of depth k by walking the level's prefix in rank-descending order. The top-ranked node is scaled by 2^k. We keep the node scales non-increasing, but as large as possible, while making sure that each next node does not overlap the previously selected nodes. If even at its original size the candidate's bounding box would overlap an already-accepted node's scaled box, the candidate is dropped.

Dropping a relatively high-ranked candidate at a coarse level is less disruptive than it sounds. Consider Arya's node in the *Game of Thrones* graph: as Figure 1 shows, it is dropped from the top layer because a higher-ranked node, Tyrion, has covered the available region. The user still sees that part of the graph as occupied, and Arya's individual node appears at a finer level where its box fits.

3.1.0.1 Spatial-hash overlap test.

A naive overlap test against every previously accepted node would dominate the cost of the filter. We instead maintain the accepted boxes in a uniform spatial hash whose cell size is chosen so that any accepted box, and any candidate's box at the level's maximum scale, both fit inside one cell on each axis. Because of this choice, two such boxes can overlap only if their center cells differ by at most one step on each axis: testing a candidate therefore reduces to scanning the accepted boxes registered in the candidate's own cell and in the eight cells immediately around it. On the layouts we tested the expected work per candidate is constant, so the overall cost is dominated by the initial sort by PageRank. In pathological cases where many accepted boxes pile into a single cell the test degrades gracefully to the

naive scan. In our experiments this phase was never a bottleneck: the per-level rerouting of Section 3.2 dominates the build time at every level we measured.

3.2 Stable Per-Level Rerouting with a Routing Hint

At each level we change the graph and the node sizes, so the edges must be rerouted on each level as well. We rerun the sleeve router, Section 4, on the set of the level edges.

3.2.0.1 Per-level CDT.

For level $z-1$ we build a dedicated CDT from the inflated polylines of V_{z-1} only, using each node's level-$z{-}1$ scale. The global landmark v_0, for example, enters the triangulation as an obstacle that is $2^{Z-z+1}\times$ its finest-level size in each dimension, so routes naturally bend around it.

3.2.0.2 Routing hint.

For every edge $e \in E_{z-1}$ we already have a route c_e computed at the finer level z. Sampling c_e and listing the level-$z{-}1$ CDT triangles it visits, expanded by one neighbor hop, yields a triangle *sleeve* S_e that we can use as a per-edge hint for an A* search restricted to S_e. This keeps the coarse route close to the fine one, so an edge deforms smoothly as the user zooms out instead of flipping to the other side of a landmark. The price is one independent A* per edge.

By default we instead reuse the per-source Dijkstra trees of Section 4.2, which are faster and behave well in practice. The reason this option is not always available is that, in a subgraph, the set of transparent obstacles is per-edge rather than per-source: the boundaries of nodes on the ancestor chains of the edge's endpoints may be crossed, and for edges sharing a source these chains differ with the target. A single Dijkstra tree from the source therefore cannot serve all of its outgoing edges in this setting, so the hint+A* variant is used in this case. The graphs listed in the paper do not have subgraphs.

3.3 Splitting Edges between Tiles

Once each level G_z has its final geometry, we cut that level into tiles. A tile is a pair $(\mathit{rect}, \mathit{tiledata})$, where *tiledata* is a set of *tile elements*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a pair (e, p), with p a continuous piece of the edge curve c_e confined to *rect*.

The single tile at level 0 is obtained by clipping every element to the graph's bounding box. To build level $z+1$ from level z, each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines into sub-clips confined to individual sub-tiles. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile's rectangle, crossing the boundary only at its endpoints; (b) the union of all clips for edge e inside a tile equals $c_e \cap \mathit{rect}$. Part (a) makes subdivision cheap and local: thanks to (a) we never need to clip against the four outer sides of a tile, only against its two midlines. For the default polyline edges produced by the sleeve router this is trivial—each segment meets each midline at most once—so a clip is split by walking its segments and cutting at the midline crossings. The optional Bezier-corner pass of Section 4.3 replaces these line–line intersections with curve–line ones plus a parameter sort, but obeys the same invariant. Part (b) is the rendering-correctness condition: at any zoom only the tiles intersecting the viewport are fetched and their clips drawn, and (b) guarantees

Table 1 Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$, $Z_{\max} = 8$). The capacity $\mathcal{C}$ only guides the choice of Z; we have no tight analytical bound on the actual rendered per-tile element count, which is therefore reported empirically. "Max tile's level" is the level on which the maximum tile was found. In the worst cases, `ca-HepPh` and `ca-CondMat`, the densest tile holds 6–8× the nominal capacity $\mathcal{C}$.

Graph	$\lvert V\rvert$	$\lvert E\rvert$	Levels built	Max per tile	Max tile's level
gameofthrones	407	2 639	5	324	4
composers	3 405	13 832	8	749	3
ca-GrQc	5 242	28 968	8	359	4
ca-HepTh	9 877	51 946	8	1 836	3
facebook_combined	4 039	88 234	9	928	5
ca-HepPh	12 008	236 978	9	3 643	3
ca-CondMat	23 133	186 878	8	2 949	3
deezer_europe	28 281	92 752	8	2 402	3
delaunay_n15	32 768	98 274	8	283	7

that the union of what is drawn equals $c_e \cap$ viewport for every visible edge e. The same invariant makes the rerouting of Section 3.2 compatible with tiling: whenever an edge curve is replaced by a coarser-level route, (b) is reestablished by recomputing the affected clips.

Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively. These are the same conditions that determine Z (Algorithm 1).

Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

4 Sleeve Routing on the CDT

We describe an edge routing method that routes edges directly on the dual graph of a Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a *sleeve*—a sequence of adjacent CDT triangles from source to target—and applies the funnel algorithm to produce the shortest path within it.

4.1 CDT Construction and the Dual Graph

Given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [16, 19]. The *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. A triangle is *free-space* if it is not contained in any obstacle interior.

4.2 Routing inside a Sleeve

Intuitively, the sleeve is a chain of triangles meeting along shared edges, the *diagonals*, that the path must cross in order. Routing an edge from s to t splits into two subproblems: a *combinatorial* one—which sequence of diagonals to cross—and a *geometric* one—which exact polyline through that sequence is shortest. The geometric step is the classical funnel algorithm [15, 29], which pulls the path taut inside the polygon formed by the sleeve triangles.

The sleeve router itself is therefore agnostic to how the sleeve is produced; any search on the dual graph that returns such a sequence of triangles will do.

To obtain the sleeve we run a shortest-path search on D where the step from one triangle to a neighbor costs the Euclidean distance between their centroids; triangles inside obstacles other than the source and target obstacles are excluded. The natural per-edge instantiation is A* [28] with the straight-line distance to t as heuristic. When a node has many incident edges, however, running independent searches is wasteful. We instead group edges by source s and run a single Dijkstra computation per source, expanding until all targets are reached and recovering each sleeve by following parent pointers. This reduces the number of searches from m to at most n, the number of nodes; on the Game of Thrones social network (407 nodes, 2 639 edges) it cuts routing time by 30%.

Since the query edges are undirected on D, each can be served from either endpoint, so the number of distinct roots is in turn reducible by reorienting query edges to minimize that count—an instance of minimum vertex cover on the demand graph, well approximated in linear time by the greedy maximum-degree rule and used as a source-side speed-up in many-to-many shortest-path frameworks [35]. Applying this heuristic reduces the number of Dijkstra trees from 331 (one per distinct source) to 199 on Game of Thrones and from 2 645 to 1 285 on the composers graph (3 405 nodes, 13 832 edges), shortening the routing pass by ∼17% and ∼35% respectively.

Before launching the sleeve search for an edge (s, t) we attempt an even cheaper shortcut: starting at the triangle containing s, we walk the triangulation along the segment $\overline{st}$ triangle by triangle and check whether any traversed triangle is interior to a third obstacle. If none is, the straight segment is the geodesic and we use it directly, skipping both the dual-graph search and the funnel.

4.3 Vertex Collapse at Source and Target

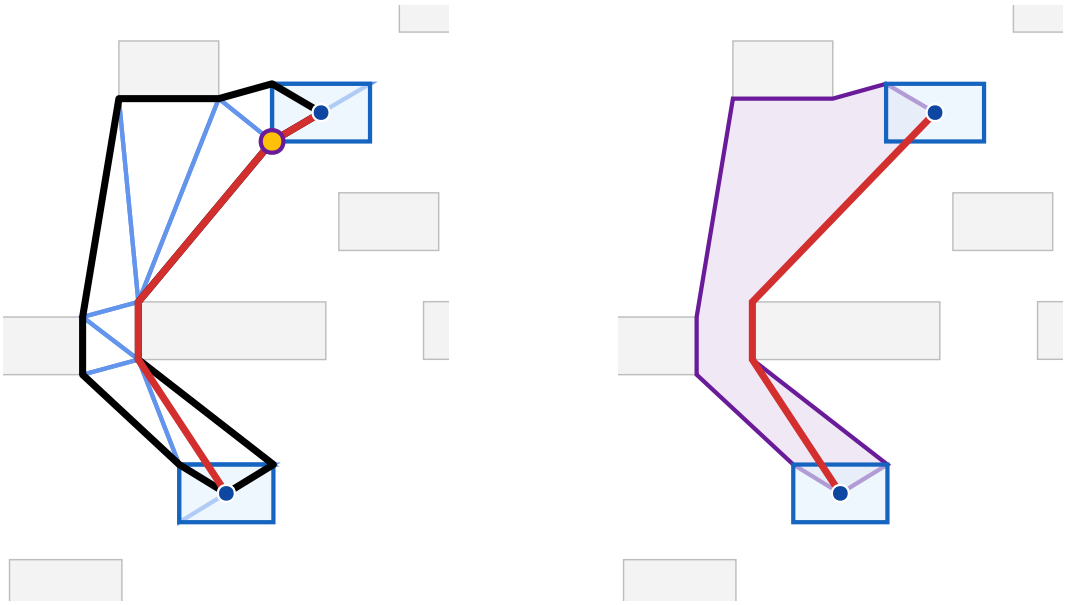
We improve the sleeve's geometry in the situation depicted in Figure 3. The remedy is to *collapse* corner-hugging chain vertices to the corresponding endpoint. Walking the right chain of the sleeve outward from s, we find the first vertex of the source obstacle at which the chain turns right toward s, and replace every source-owned vertex on that chain up to and including this one by s. The left chain is processed mirror-symmetrically (with left turns instead of right), and the target end likewise. Diagonals whose two endpoints coincide after collapse are dropped, and the left/right orientation of each surviving diagonal is preserved from the uncollapsed geometry. The collapsed vertex in Figure 3(a) is marked in yellow.

After collapse, the funnel sees a wide opening at each endpoint, as in Figure 3(b), and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves. Collapse might cause the resulting path to overlap nodes other than the source and the target, but we have found the effect to be negligible.

The implementation also offers an optional pass that fits a Bezier segment into each polyline corner to render edges as smooth curves. It is disabled by default for performance: the polyline rendering is fast and visually acceptable, so we trade the smoother curves for routing speed.

5 WebGL Rendering

The WebGL renderer uses deck.gl [2], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView`, with no perspective distortion, and a `TileLayer` that dynamically loads tile data based on the current viewport. Because



(a) before collapse (b) after collapse

Figure 3 Effect of vertex collapse on the LORAS–RENLY edge of the Game of Thrones graph. Endpoints are shown as blue dots; their padded obstacles are highlighted in light blue. (a) The CDT triangles forming the sleeve are drawn in cornflower blue and the funnel path in red; the thick black polyline is the perimeter of the non-collapsed sleeve. The yellow dot marks the source-obstacle vertex that gets collapsed. (b) The sleeve after collapse, drawn as a single purple polygon with the source/target obstacles merged into the corresponding endpoints, and the funnel path (red) through it. Collapsing widens the funnel opening at the endpoint and removes the spurious detour around the obstacle corner.

deck.gl's `TileLayer` expects a power-of-two pyramid of 512-pixel tiles, the root tile size is rounded up to the nearest power of two and a *start zoom* level is computed so that the root tile maps to deck.gl's 512-pixel tile size. The `TileLayer`'s `getTileData` callback then translates deck.gl tile coordinates (x, y, z) to MSAGLJS tile map coordinates, fetching the precomputed tile data.

In deck.gl's `OrthographicView` the screen scale at view zoom v is $s = 2^v$ pixels per world unit, and the start zoom is $v_0 = \log_2(512/W)$, so that at $v = v_0$ the root tile of width W projects to 512 pixels. The `TileLayer` then requests pyramid level $z = \lfloor v - v_0 \rfloor$, at which each tile covers $W/2^z$ world units and renders at about 512 pixels. Note v_0 lives in the same view-zoom space as v and is not a pyramid level: when the bounding box is bigger than one 512-pixel tile, v_0 is negative. For example, with a bounding box of 1000×800 world units rounded up to $W = 1024$ we get $v_0 = -1$; at $v = 2$ the scale is $s = 4$ px/wu, the chosen level is $z = 3$, and each level-3 tile spans 128 wu. A 1280×800-pixel viewport then covers 320×200 wu and intersects $\lceil 320/128 \rceil \times \lceil 200/128 \rceil = 3 \times 2 = 6$ level-3 tiles, the only ones fetched (Figure 1). Between integer view zooms the level is held fixed and the tiles are GPU-scaled, so continuous panning and zooming never block on routing or rerouting.

5.0.0.1 Rendering layers.

Each tile is rendered as a composite deck.gl layer containing:

- A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).

Graph	$\lvert V\rvert$	$\lvert E\rvert$	Routing	Tiling	Total
gameofthrones	407	2 639	0.17	0.17	1.87
composers	3 405	13 832	4.46	2.43	7.67
ca-GrQc	5 242	28 968	4.96	2.72	9.16
ca-HepTh	9 877	51 946	27.60	9.62	40.51
ca-HepPh	12 008	236 978	83.40	35.61	128.10
facebook_combined	4 039	88 234	13.82	6.55	22.33
ca-CondMat	23 133	186 878	202.50	55.29	273.60
deezer_europe	28 281	92 752	388.60	68.90	479.10
delaunay_n15	32 768	98 274	31.23	12.94	68.40

Table 2 End-to-end loading benchmarks on an Apple M4 Max laptop, all times in seconds. *Routing* sums the CDT-construction and Dijkstra-tree sleeve-search phases of Section 4.2 on the full graph; *Tiling* is the build of the tile pyramid (capacity $\mathcal{C} = 500$, $Z_{\max} = 8$) of Section 3, which includes the per-level rerouting; *Total* adds parsing.

- A **TextLayer** for node labels.
- A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at zoom level z, the resolution is 2^{z-2} segments per curve span.
- An **IconLayer** for edge arrowheads.
- A **TextLayer** for edge labels.

5.0.0.2 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

6 Experimental Results

We evaluate the sleeve router and the tile-pyramid build on the same benchmark graphs as Table 1: the Game of Thrones social network [13], the GD 2011 contest `composers` graph [9], four SNAP collaboration networks (`ca-GrQc`, `ca-HepTh`, `ca-HepPh`, `ca-CondMat`) and the SNAP `facebook_combined` ego network [37, 3], the `deezer_europe` social network [42], and the SuiteSparse Delaunay mesh `delaunay_n15`. All experiments run in Node.js (single process, headless; not a browser) on an Apple M4 Max laptop with 48 GB of unified memory. Layout uses IPsepCola, and the tile pyramid is built with the same parameters as Table 1, capacity $\mathcal{C} = 500$ and $Z_{\max} = 8$.

Table 2 reports per-graph timings of three pipeline stages: *Routing*, the sum of the two phases logged by the sleeve router on the full graph—constrained Delaunay triangulation of the obstacles and the Dijkstra-tree search on the CDT dual (Section 4.2); *Tiling*, the build of the tile pyramid (Section 3), which includes the per-level rerouting of Section 3.2; and *Total*, the end-to-end loading time including parsing.

The CDT phase contributes negligibly to the routing column—from 8 ms on Game of Thrones to about half a second on the largest graphs—so virtually all of *Routing* is the Dijkstra-tree sleeve search, with the vertex-cover heuristic of Section 4.2 reducing the number of trees we run. Even on graphs with hundreds of thousands of edges (`ca-CondMat`, `ca-HepPh`, `deezer_europe`) the full pipeline finishes in a few minutes; this is a one-time build cost. The resulting tile pyramid already contains the per-level routes of Section 3.2, so panning and zooming only fetch and render the precomputed tiles, with no rerouting at view time.

7 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The sleeve routing method routes edges directly on the CDT dual graph using batched Dijkstra trees and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at `https://github.com/microsoft/msagljs` under the MIT license.

References

1. Cosmograph. `https://cosmograph.app`.
2. deck.gl: Large-scale WebGL-powered data visualization. `https://deck.gl/`.
3. facebookcombined. `https://snap.stanford.edu/data/facebook_combined.txt.gz`.
4. Funnel algorithm. `https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf`.
5. Graphviz. `http://www.graphviz.org/`.
6. Regraph. `https://cambridge-intelligence.com/regraph/`.
7. sfdp. `https://graphviz.org/docs/layouts/sfdp/`.
8. Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. `https://www.sigmajs.org/`.
9. Skewed. `http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml`.
10. TileLayer (`@deck.gl/geo-layers`). `https://deck.gl/docs/api-reference/geo-layers/tile-layer`. Accessed: 2026.
11. yworks. `https://yworks.com/products/yed`.
12. James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.
13. Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.
14. Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18-20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.
15. Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.
16. Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.
17. Douglas Demyen and Michael Buro. Efficient triangulation-based pathfinding. In *Proceedings of the 21st National Conference on Artificial Intelligence (AAAI)*, pages 942–947, 2006.
18. David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.
19. Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.
20. Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. `https://arxiv.org/abs/2504.17033`.
21. Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):821–828, 2006. `doi:10.1109/TVCG.2006.156`.

22 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.

23 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016.

24 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468, 2005.

25 Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000.

26 Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan. Linear-time algorithms for visibility and shortest path problems inside triangulated simple polygons. *Algorithmica*, 2(1–4):209–233, 1987.

27 David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.

28 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.

29 John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994.

30 John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.

31 Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.

32 Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization. In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.

33 Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–71, 2005.

34 Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library, 2012.

35 Sebastian Knopp, Peter Sanders, Dominik Schultes, Falk Schieferdecker, and Dorothea Wagner. Computing many-to-many shortest paths using highway hierarchies. In *Proceedings of the 9th Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 36–45. SIAM, 2007.

36 D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 14(3):393–410, 1984.

37 Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es, 2007.

38 Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd, and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA, September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.

39 Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.

40 Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.

41 Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.

471 **42** Benedek Rozemberczki and Rik Sarkar. Characteristic Functions on Graphs: Birds of a
472 Feather, from Statistical Descriptors to Parametric Models. In *Proceedings of the 29th ACM*
473 *International Conference on Information and Knowledge Management (CIKM '20)*, page
474 1325–1334. ACM, 2020.
475 **43** Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation.
476 *Computational Geometry*, 22(1–3):21–74, 2002.
477 **44** Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding
478 of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*,
479 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.
480 **45** Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture*
481 *Notes in Computer Science*, 5849:219–231, 2010.